**UNIVERSIDAD AUTÓNOMA DE BUCARAMANGA**

Facultad de Ingeniería

Ingeniería de Sistemas

# INFORME DE LABORATORIO 5

## Visualización y alertas: MQTT, InfluxDB y Grafana en una máquina virtual

Presentado por:

Juan Sebastian Lizcano Jaimes

Angie Katherine Suarez Ortiz

Miguel Angel Hernandez Quintero

IoT + Cloud + Sistemas Distribuidos

Bucaramanga, octubre de 2026

**Resumen**

Se montó una plataforma propia en la VM de Azure del Laboratorio 2 con Docker Compose. Un publicador envía temperatura, humedad e iluminancia por MQTT a Mosquitto; una ingesta las guarda en InfluxDB 2 con su marca de tiempo real, y Grafana OSS, como servicio aparte, solo lee InfluxDB. La toma de 4 días fue del 10/10/2026 13:52:29 al 14/10/2026 13:52:29 y la alerta se disparó y se resolvió con evidencia. Las capturas están en el Anexo A.

## 1. Arquitectura: protocolo, TSDB y Grafana

La estación estacion-01 mide cada 10 s y publica un JSON con las tres variables y la hora de medición en lab5/estacion-01/telemetria, con QoS 1. Mosquitto 2.0.22 exige usuario y clave y guarda en disco los mensajes QoS 1. La ingesta está suscrita a lab5/+/telemetria con sesión persistente y solo confirma cada mensaje cuando InfluxDB 2.9.1 ya aceptó la escritura. Cada lectura queda como un punto de la medición ambiente, en el bucket lab5, con la hora del publicador y no con la de llegada. Grafana 13.2.3 consulta InfluxDB con un datasource Flux y no participa en la ingesta. Los seis servicios están en un solo docker-compose.yml con reinicio automático. Mosquitto, InfluxDB y Grafana solo escuchan en 127.0.0.1, y el operador entra por túnel SSH, sin abrir puertos en Azure.

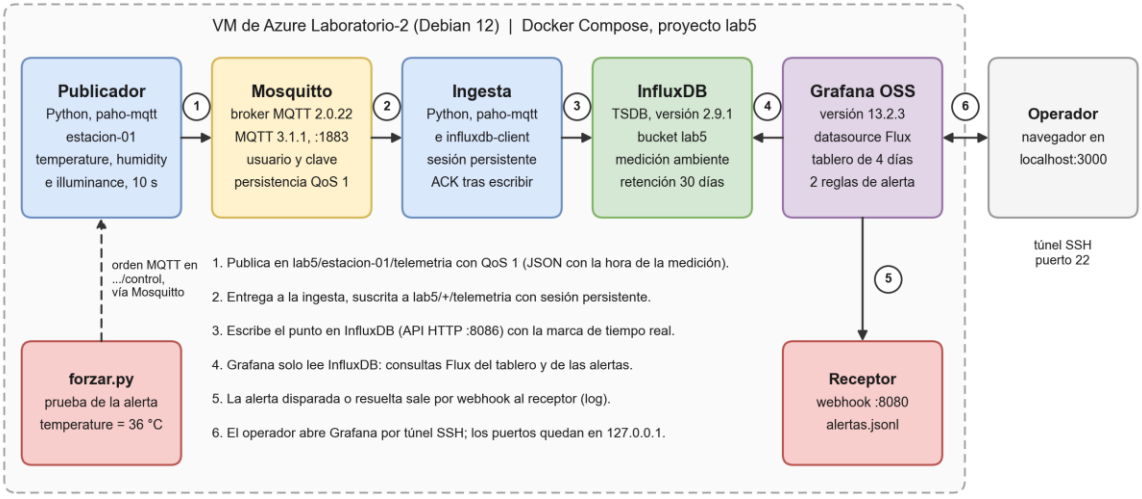



Figura 1. Flujo del protocolo elegido (MQTT) a la TSDB (InfluxDB) y a Grafana en la VM.

## 2. Justificación del protocolo

Se eligió MQTT 3.1.1 con QoS 1. Es el protocolo que ya usan el ESP32 y la VM desde el Laboratorio 2, así que el mismo firmware puede publicar en esta plataforma con solo cambiar el broker. El modelo publicar/suscribir separa al sensor de quien guarda los datos, el encabezado fijo mínimo es de 2 bytes y Mosquitto ocupa 4 MB de RAM en la VM. Un broker AMQP como RabbitMQ suele necesitar más de 100 MB, y CoAP no trae un broker con sesiones persistentes, así que la cola habría que programarla en el servidor. Para comprobar que la toma no pierde lecturas se hicieron dos pruebas de 40 s cada una: primero se detuvo la ingesta y después el broker. En ambos casos llegaron todas las lecturas, con retrasos de 2 a 52 s (Tabla A1).

## 3. Alerta de umbral

| Elemento | Configuración | Resultado de la prueba (10/10/2026) |
|---|---|---|

| Elemento | Configuración | Resultado de la prueba (10/10/2026) |
|---|---|---|
| **Condición** | Promedio de temperature de estacion-01 mayor que 32 °C | Promedio de 36,0 °C con el valor forzado |
| **Ventana** | Últimos 2 minutos, evaluada cada 30 s, con 1 minuto en pendiente | Pendiente 13:58:55, disparada 13:59:55 y normal 14:05:25 |
| **Acción** | Webhook al receptor de alertas, que guarda alertas.jsonl, y marca en el panel de temperatura | FIRING recibido a las 14:00:05 y RESOLVED a las 14:06:05 |

El umbral de 32 °C está por encima del máximo normal del escenario (18 a 30 °C), así que solo se alcanza con una falla real de ventilación. Para disparar la alerta se le ordenó al publicador forzar 36 °C durante 6 minutos con forzar.py, a las 13:57:44. Esas lecturas quedan marcadas con forzado = 1 en InfluxDB. Hay además una segunda regla, "Toma sin datos", que se dispara si pasan 5 minutos sin lecturas, para vigilar los huecos durante los 4 días.

### 4. Tablero y ventana de 4 días

El tablero "Laboratorio 5 - Estación ambiental junto al panel solar" abre por defecto con los últimos 4 días. Tiene una línea que dice qué mira el operador y tres medidores con la unidad y el rango del escenario: temperatura de 0 a 50 °C, humedad de 0 a 100 % e iluminancia de 0 a 100 000 lux. También muestra el último dato recibido, el inicio de la toma, una serie de tiempo por variable con el umbral en rojo, los mensajes guardados por hora (se esperan 360) y el estado de las alertas.

| Dato | Valor (hora de Colombia) |
|---|---|
| **Inicio de la toma** | 10/10/2026 13:52:29 |
| **Fin de la ventana** | 14/10/2026 13:52:29 |
| **Cierre de la toma** | 14/10/2026 13:52:29, con 34560 de 34560 lecturas esperadas (100,00 %) |
| **Huecos en los 4 días** | Ninguno mayor a 30 s; 36 lecturas forzadas por la prueba de la alerta |
| **Registro de la toma** | Registro diario por cron a las 23:59 y consulta final del 14/10 (evidencias/registros) |

La toma completó la ventana de 4 días. La Figura A15 muestra el tablero con el rango absoluto de los 4 días, y evidencias/registros/consulta_4_dias.txt guarda la consulta final de InfluxDB.

### 5. Qué falló al instalar en la VM

- **Docker.** No estaba instalado; se usó el repositorio oficial de Docker (Engine 29.9 y Compose v5.6).
- **swapon.** El script de instalación consultó swapon sin sudo, y en Debian ese comando está en /sbin, fuera del PATH del usuario: salió "swapon: command not found". El swap se creó igual, porque la verificación devolvió cero líneas; luego el script se corrigió con sudo.
- **Memoria.** La VM tiene 903 MB de RAM y no tenía swap. Con el stack arriba, Grafana 13 usa unos 340 MB y quedaban 138 MB libres. Se agregaron 2 GB de swap y se limitó la caché de InfluxDB a 100 MB.
- **Primer mensaje.** La primera lectura salió antes de que el broker aceptara la conexión y se perdió; ahora el publicador espera la conexión.
- **cron y Grafana.** La imagen de Debian de Azure no trae cron y se instaló. Grafana registraba un error por la carpeta de provisioning de plugins, que se agregó vacía.

## Anexo A. Evidencias

Las capturas siguen el orden en que se hizo el laboratorio. Las de terminal son de la sesión SSH a la VM y las de Grafana se tomaron a través del túnel SSH. Los archivos están en evidencias/capturas, y los registros completos (instalación, docker compose up, prueba de resiliencia, inicio de la toma, alertas recibidas y consultas) en evidencias/registros.

### A.1 Instalación y stack en la VM

Instalación de Docker con scripts/instalar_vm.sh. Se ven las versiones, el error de swapon y el swap de 2 GB ya activo.

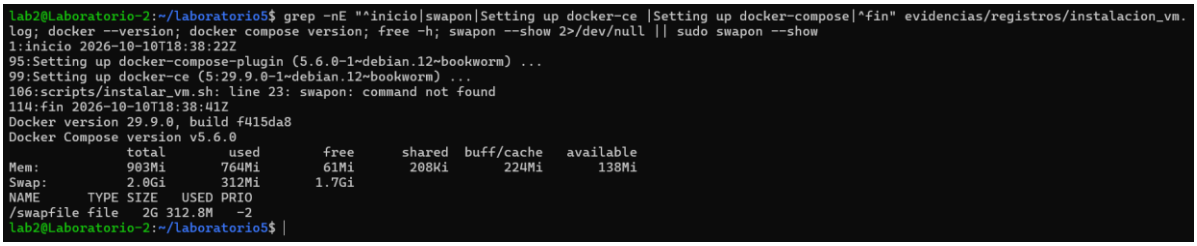



Figura A1. Docker y Compose instalados, error de swapon y memoria de la VM.

Los seis servicios del docker-compose.yml en ejecución y el consumo de memoria de cada contenedor.

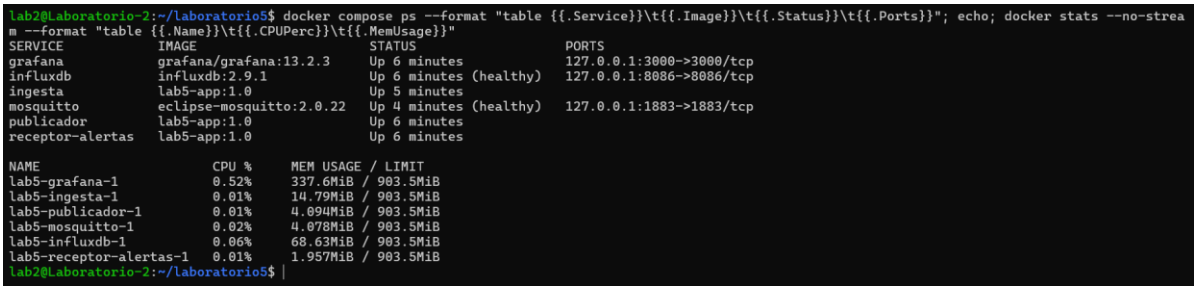



Figura A2. docker compose ps y docker stats.

### A.2 Protocolo MQTT y TSDB

Mensajes en el topic lab5/estacion-01/telemetria leídos con mosquitto_sub dentro del broker, con las tres variables, y logs del publicador y de la ingesta.

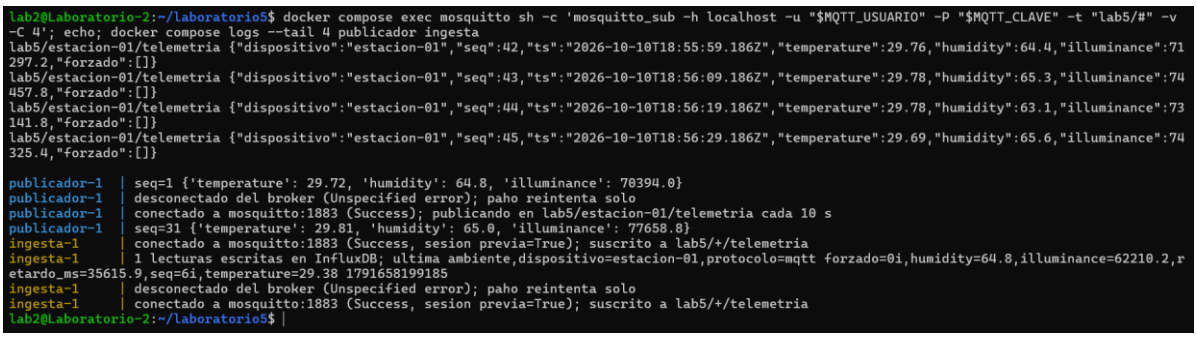



Figura A3. Publicación MQTT con temperature, humidity e illuminance y escritura en InfluxDB.

Consulta de los puntos guardados con consultar.py (inicio, puntos, huecos y últimas lecturas) y directamente con influx query.

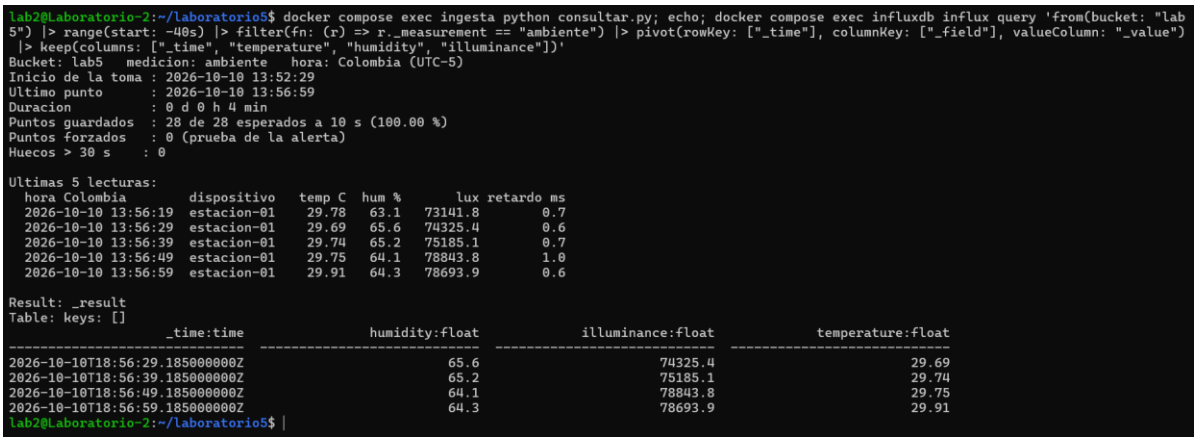



Figura A4. Consulta de los puntos guardados en InfluxDB.

Antes del inicio oficial de la toma se probó la entrega con caídas de 40 s de la ingesta y del broker (evidencias/registros/prueba_resiliencia.log):

| **Prueba** | **Lecturas afectadas** | **Retraso al llegar** | **Pérdidas** |
|---|---|---|---|
| Ingesta detenida 40 s | 4 (seq 6 a 9) | 36, 26, 16 y 6 s | 0: el broker las guardó |
| Broker detenido 40 s | 6 (seq 12 a 17) | 52 a 2 s | 0: paho las guardó |

Tabla A1. Prueba de resiliencia de la entrega MQTT con QoS 1.

## A.3 Tablero en Grafana

Tablero al inicio de la toma, abierto como Viewer: la línea para el operador, los medidores con unidad y rango, las series y el estado de las alertas.

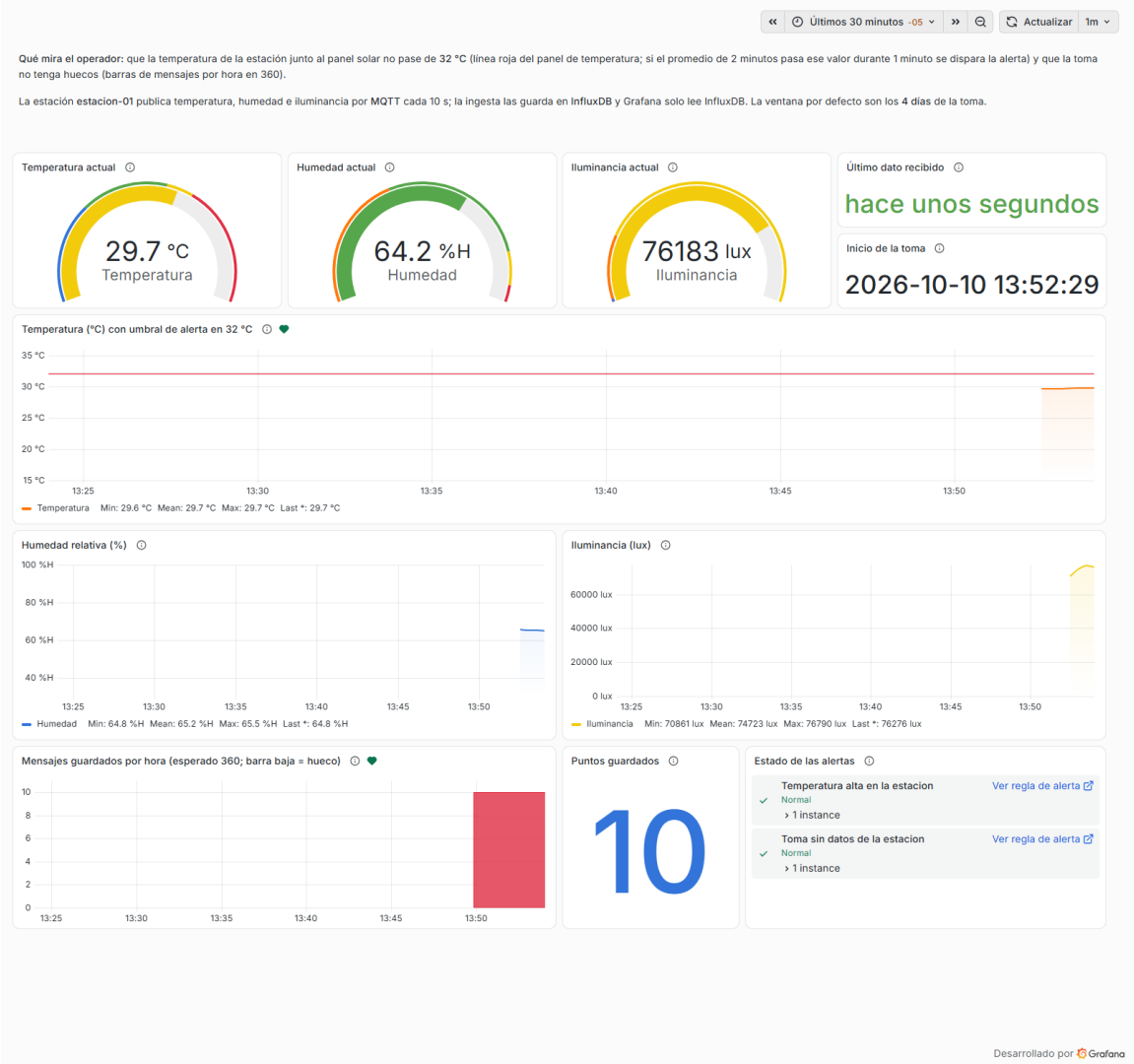



Figura A5. Tablero de la estación al inicio de la toma.

## A.4 Prueba de la alerta de umbral

Orden de forzado desde el publicador: temperature = 36 °C durante 6 minutos.

```
lab2@Laboratorio-2:~/laboratorio5$ date; docker compose exec publicador python forzar.py temperature 36 --minutos 6; sleep 12; docker compose logs --s
ince 25s publicador
Sat Oct 10 18:57:44 UTC 2026
orden enviada a lab5/estacion-01/control: {'variable': 'temperature', 'valor': 36.0, 'duracion_s': 360.0}
publicador-1  | FORZADO temperature=36 durante 360 s
publicador-1  | seq=53 {'temperature': 36.0, 'humidity': 64.7, 'illuminance': 79817.4} forzado=['temperature']
lab2@Laboratorio-2:~/laboratorio5$
```

Figura A6. forzar.py y log del publicador con las lecturas forzadas.

Con la alerta disparada: el medidor en rojo, la serie sobre la línea de 32 °C con las marcas de pendiente y disparada, y la regla en Firing.

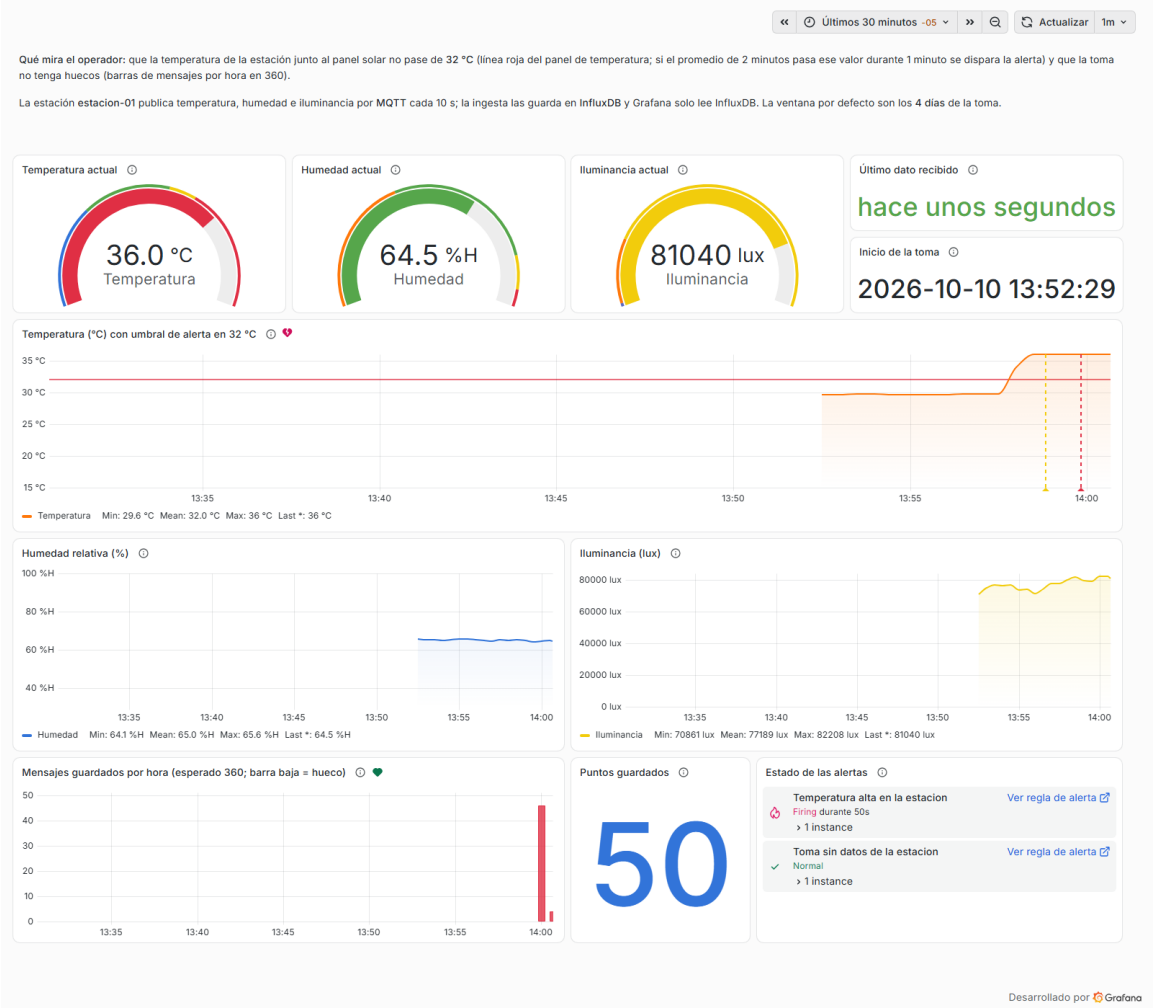



Figura A7. Tablero durante la alerta.

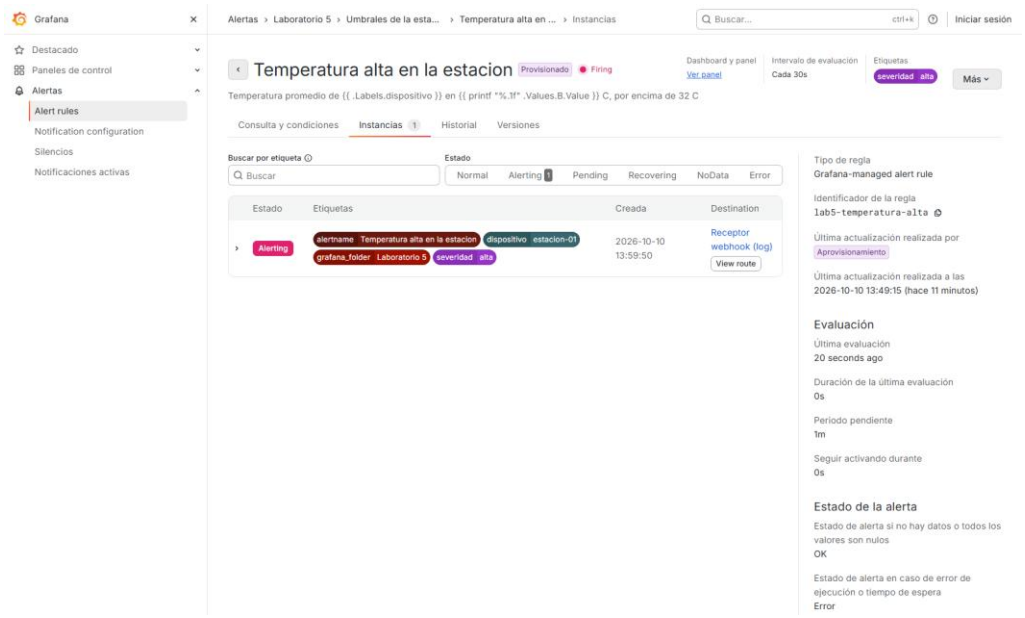



Figura A8. Instancia de la regla en estado Alerting y su punto de contacto.

```
lab2@Laboratorio-2:~/laboratorio5$ docker compose logs receptor-alertas; echo; tail -n 1 datos/alertas.jsonl | python3 -m json.tool | grep -E "\"(stat
us|alertname|dispositivo|severidad|summary|startsAt|valueString|receiver)\"" | head -12
receptor-alertas-1  | receptor de alertas escuchando en :8080, guarda en /datos/alertas.jsonl
receptor-alertas-1  | 2026-10-10T19:00:05Z FIRING Temperatura alta en la estacion dispositivo=estacion-01 valores=[ var='B' labels={dispositivo=estaci
on-01} type='reduce' value=36 ], [ var='C' labels={dispositivo=estacion-01} type='threshold' value=1 ]

        "receiver": "Receptor webhook \\(log\\)",
        "status": "firing",
            "status": "firing",
                "alertname": "Temperatura alta en la estacion",
                "dispositivo": "estacion-01",
                "severidad": "alta"
                "summary": "Temperatura promedio de estacion-01 en 36.0 C, por encima de 32 C"
            "startsAt": "2026-10-10T18:59:50Z",
            "valueString": "[ var='B' labels={dispositivo=estacion-01} type='reduce' value=36 ], [ var='C' labels={dispositivo=estacion-01} type='
threshold' value=1 ]",
          "alertname": "Temperatura alta en la estacion",
          "alertname": "Temperatura alta en la estacion",
          "dispositivo": "estacion-01",
lab2@Laboratorio-2:~/laboratorio5$
```

Figura A9. Notificación FIRING recibida por el webhook y su contenido en alertas.jsonl.

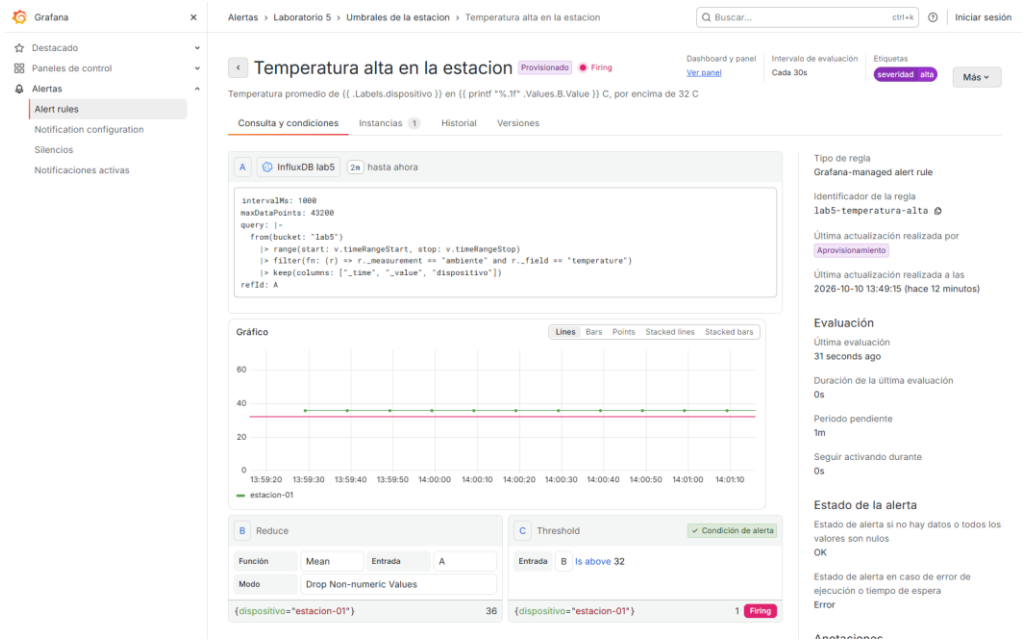



Figura A10. Consulta de la regla, ventana de 2 minutos, reducción por promedio y umbral de 32 °C.

Al terminar el forzado la temperatura vuelve a unos 30 °C, la regla regresa a Normal y Grafana envía el aviso de resolución.

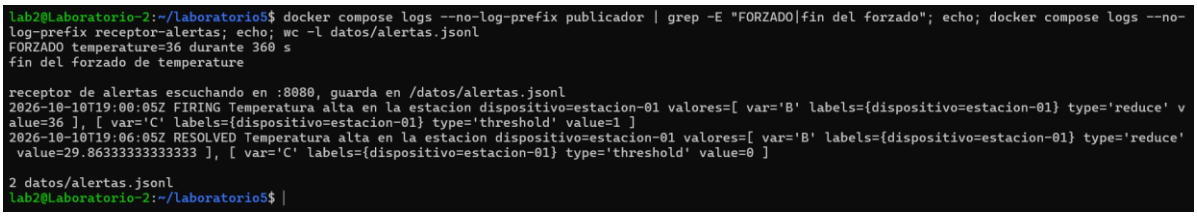



Figura A11. FIRING y RESOLVED en el receptor de alertas.

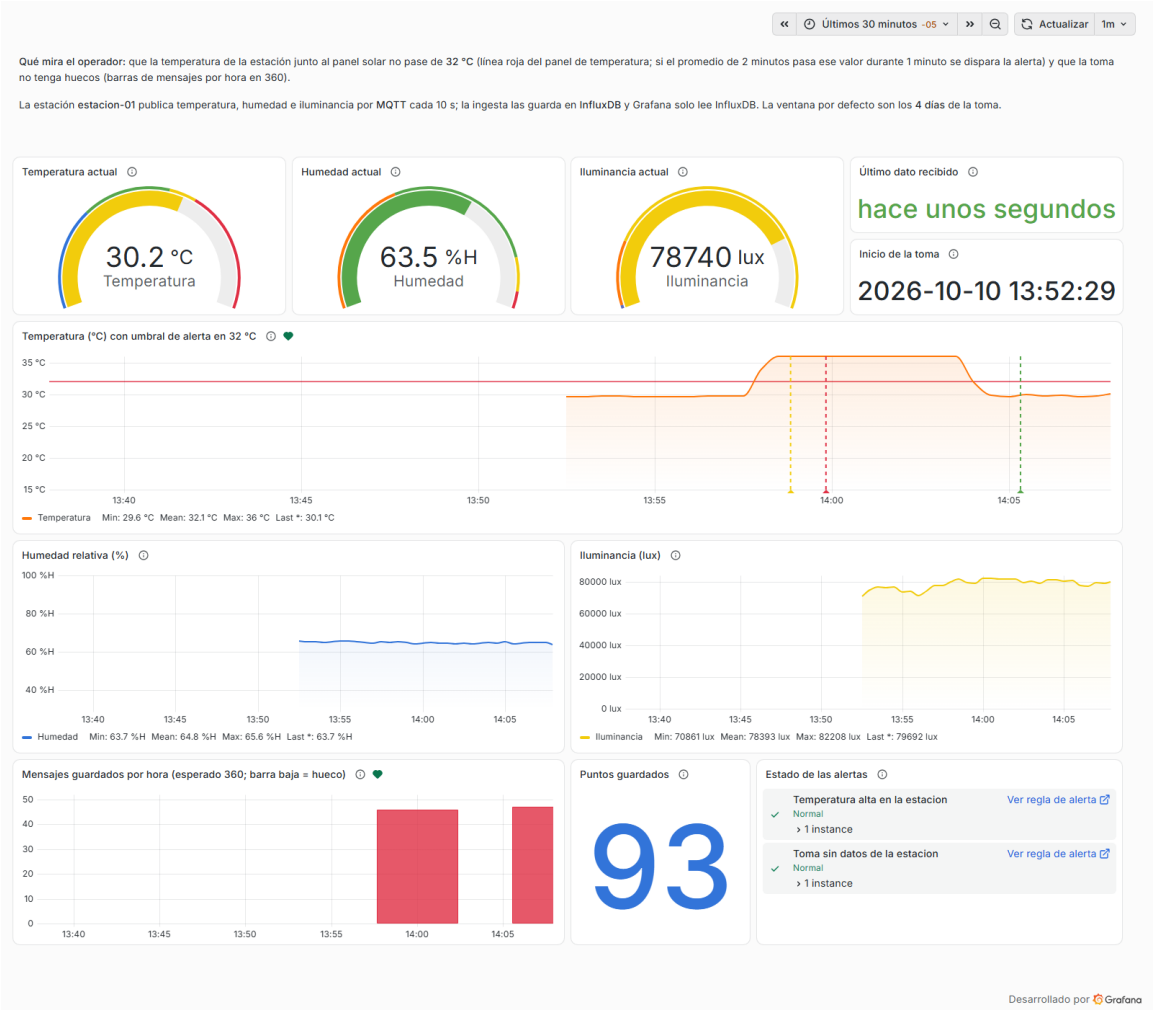



Figura A12. Tablero después de la prueba, con las marcas Pendiente, Disparada y Normal.

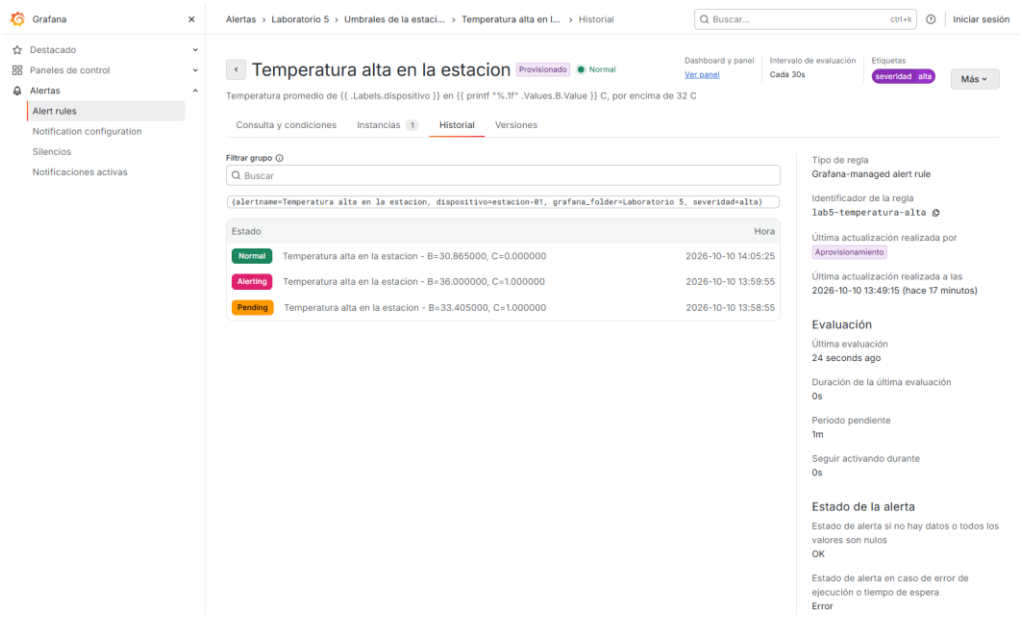



Figura A13. Historial de estados de la regla.

## A.5 Ventana de la toma: corte y cierre de los 4 días

Tablero con el rango absoluto desde el inicio de la toma hasta la fecha de corte de este informe.

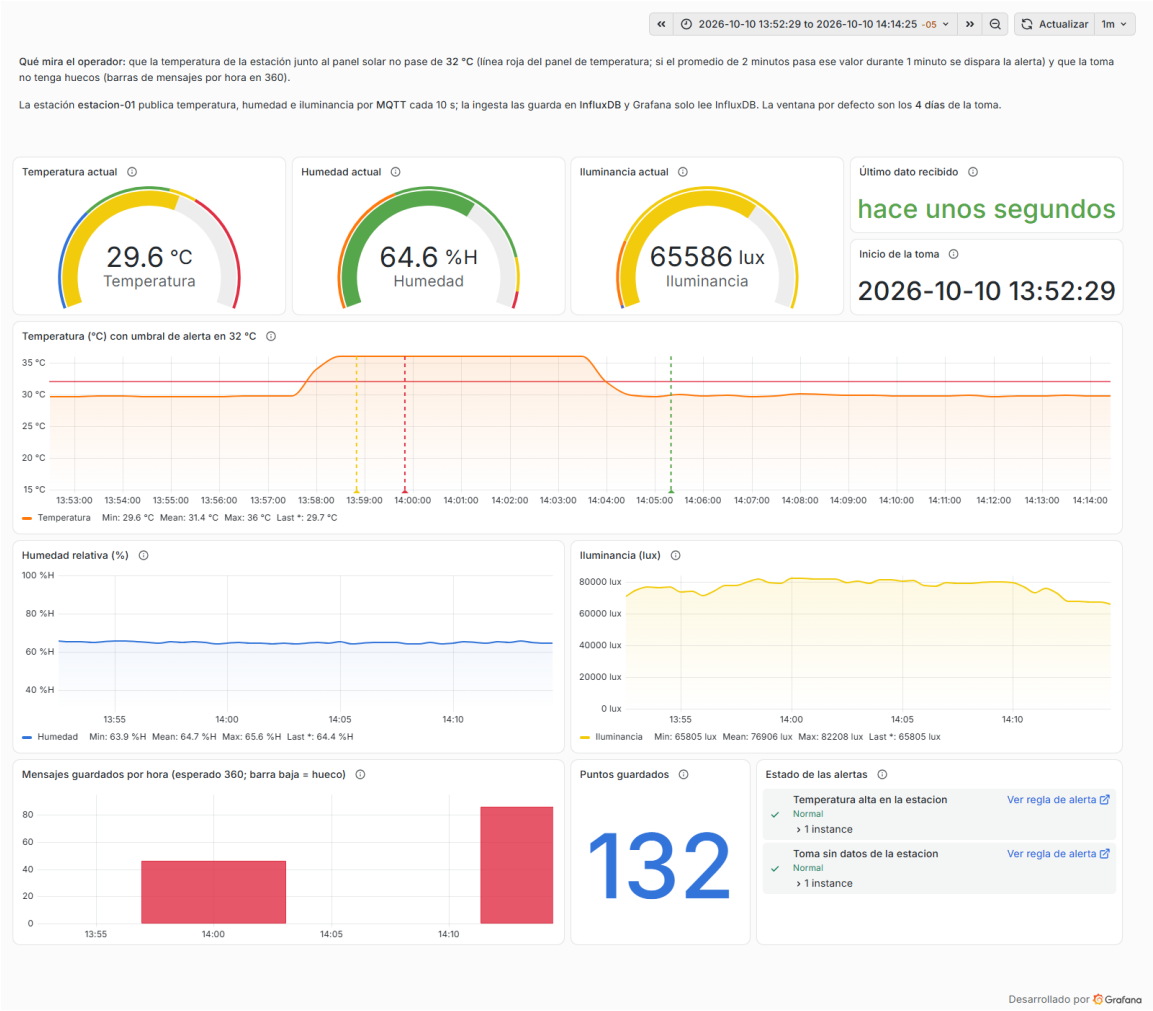



Figura A14. Toma del 10/10/2026 13:52:29 al corte de las 14:14:19.

Tablero con el rango absoluto de los 4 días completos de la toma.

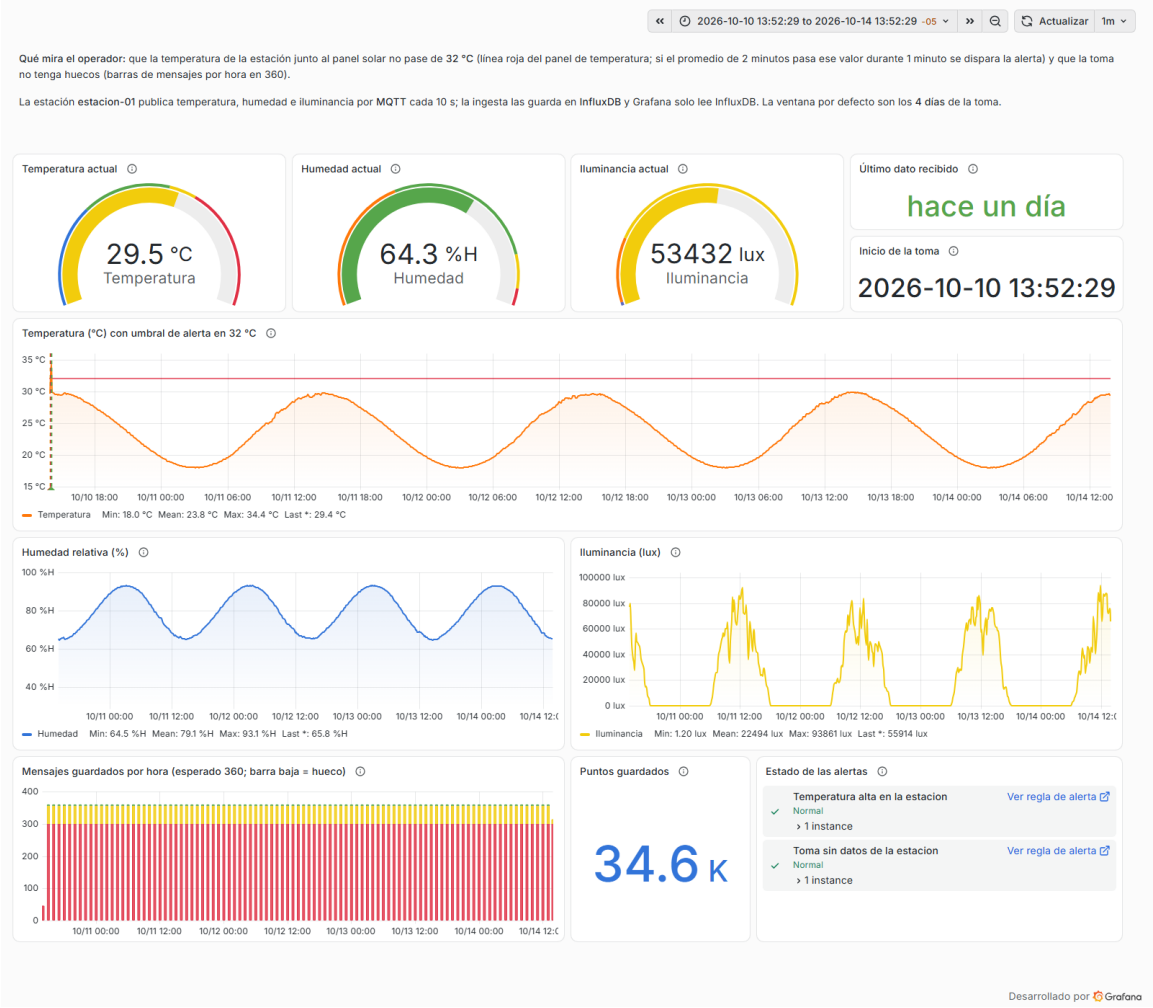



Figura A15. Toma completa del 10/10/2026 13:52:29 al 14/10/2026 13:52:29.

**Repositorio GitHub**

https://github.com/Hypnostian/IOT.git (carpeta Laboratorio 5)